Documentación del Proyecto Compilador
Etapa 4: Generación de Código Intermedio y Testing

1. División del Trabajo

El desarrollo de la cuarta etapa del compilador, centrada en el diseño e implementación del generador de Código Intermedio de Tres Direcciones (`codinter`), su integración con el gestor de compilación y la elaboración de pruebas unitarias, se realizó de manera colaborativa distribuyendo las responsabilidades de la siguiente forma:

- **Diseño e implementación del Generador de codigo intermedio (`codinter.c/.h`):** Gribaudo Re Francisco y Francisco Andreani.
- **Manejo del compilador y CLI para integrar el "codinter" (`compiler-manager.c/.h`, `cli.c`):** Valentín Pastre Thuer.
- **Elaboración de pruebas unitarias (`test_codinter.c`):** Francisco Andreani y Gribaudo Re Francisco.

2. Decisiones de Diseño, Suposiciones y Aclaraciones

Habiendo completado en la etapa anterior la validación semántica sobre el Árbol de Sintaxis Abstracta (AST) y la Tabla de Símbolos, el objetivo principal de esta etapa consistió en traducir el árbol sintáctico validado a una representación intermedia lineal de Código de Tres Direcciones, concluyendo así la construcción del *front-end* del compilador.

- **Generación Dinámica de Temporales y Etiquetas:** Se implementaron las funciones auxiliares estáticas `newTemp(enum types type)` para instanciar variables temporales locales numeradas secuencialmente (`t0, t1, t2...`) tipadas según la operación realizada, y `newLabel()` para crear etiquetas de salto (`L0, L1, L2...`) asociadas a nodos de tipo `blankNode`.
- **Carga Explícita de Constantes:** Al evaluar literales (`intLiteral`, `boolLiteral`, `floatLiteral`) en `generateExprCodinter`, se decidió emitir una instrucción `Codinter_ASSIGN` hacia un nuevo temporal antes de utilizarlos en operaciones compuestas, uniformizando el manejo de operandos en las cuádruplas.

3. Diseño y Decisiones Clave: El Generador de Código Intermedio

El núcleo de esta entrega reside en `src/codinter.c` y `header/codinter.h`, donde se definieron las operaciones del enumerado `CodinterOp` y los algoritmos de recorrido sobre el AST:

- **Emisión y Gestión de la Lista Lineal (`initCodinterGenerator`, `emit`, `getCodinterListHead`):**
  - La función `emit` reserva memoria dinámicamente para cada nueva estructura `CodinterInstruction` y la inserta al final de la lista enlazada en tiempo constante $O(1)$ utilizando `codinterTail`.
  - La función `initCodinterGenerator` reinicia los punteros `codinterHead` y `codinterTail` a `NULL` y pone en cero los contadores `tempCounter` y `labelCounter`, permitiendo ejecutar múltiples compilaciones o pruebas unitarias sin arrastrar estado previo.

- **Traducción de Expresiones (`generateExprCodinter`):** Recorre recursivamente un subárbol de expresión, emite las instrucciones correspondientes y retorna el puntero `Symbol*` donde se aloja el resultado:
    - *Identificadores y Parámetros:* Los nodos con tipo de símbolo `variable`, `global_var`, `local_var` y `formalParameter` devuelven directamente su `Symbol*` sin emitir instrucciones adicionales.
    - *Operadores Unarios (`Codinter_NEG, Codinter_NOT`):* Se distingue la resta unaria verificando cuando un nodo `substractOp` carece de hijo derecho (`expr->right == NULL`), emitiendo `Codinter_NEG` hacia un nuevo temporal. Por su parte, el nodo `negationOp` evalúa su operando izquierdo y emite `Codinter_NOT` hacia un temporal de tipo `tBoolean`.
    - *Operadores Binarios Aritméticos, Relacionales y Lógicos:* Se evalúan recursivamente ambos operandos (`leftOp` y `rightOp`), se determina el tipo del temporal resultante (`tBoolean` para operaciones relacionales y lógicas, o numérico para las aritméticas) y se emite la cuádrupla con el código correspondiente (`Codinter_ADD`, `Codinter_SUB`, `Codinter_MUL`, `Codinter_DIV`, `Codinter_MOD`, `Codinter_LT`, `Codinter_GT`, `Codinter_EQ`, `Codinter_AND`, `Codinter_OR`).
    - *Invocación de Métodos (`functionInvoke`):* Recorre la lista de argumentos actuales (`actualParameterList`), evalúa cada expresión mediante `generateExprCodinter` y emite una instrucción `Codinter_PARAM` por cada parámetro. Finalmente emite `Codinter_CALL`, asignando un temporal de destino si la función retorna un valor no vacío (`sym->type != tVoid && sym->type != blank`).
- **Traducción de Sentencias y Control de Flujo (`generateCodinterFromAST`):**
    - *Delimitación de Funciones (`function`):* Emite la directiva `Codinter_FUNC_BEGIN` al iniciar la declaración del método, recorre el cuerpo de la función (`node->right`) y finaliza emitiendo `Codinter_FUNC_END`.
    - *Asignaciones (`assignOp`):* Evalúa la expresión del lado derecho con `generateExprCodinter(node->right)` y emite `Codinter_ASSIGN` hacia el símbolo de la variable ubicada en `node->left->symbolData`.
    - *Condicionales (`ifNode`):* Evalúa la condición y emite un salto condicional `Codinter_IF_FALSE` hacia `labelFalse`. Luego traduce el bloque *then* (`ifBody->left`); si existe una rama *else* (`ifBody->right != NULL`), emite un salto incondicional `Codinter_GOTO` hacia `labelEnd`, posiciona la etiqueta `labelFalse` (`Codinter_LABEL`), traduce el bloque del `elseNode` (`ifBody->right->left`) y cierra con la etiqueta `labelEnd`.
    - *Ciclos (`whileNode`):* Crea las etiquetas `labelStart` y `labelEnd`, emitiendo primero `labelStart` (`Codinter_LABEL`) antes de evaluar la condición del bucle. Emite `Codinter_IF_FALSE` hacia `labelEnd`, procesa el cuerpo del ciclo (`node->right`), añade un salto incondicional `Codinter_GOTO` hacia `labelStart` y ubica `labelEnd` al final.
    - *Sentencias de Retorno (`returnNode`):* Si el nodo posee una expresión de retorno (`node->left != NULL`), evalúa su resultado y emite `Codinter_RETURN` con dicho operando; en caso contrario, emite `Codinter_RETURN` sin argumentos.

4. Integración con el Resto del Compilador

Para conectar el generador de código intermedio con la arquitectura general del compilador se actualizaron los siguientes módulos:

- **Gestor de Compilación (`compiler-manager.c/.h`):** Se incluyó la cabecera `codinter.h` y se implementó la función `codinterStage()`. Esta etapa ejecuta secuencialmente el parser (`yyparse()`) y el analizador semántico (`analyzeAst(syntaxTree, st)`); si ambos finalizan sin errores, invoca a `initCodinterGenerator()`, `generateCodinterFromAST(syntaxTree)` y

`printCodinter(outputFilePtr)`, escribiendo el resultado en el archivo con extensión `.ci` generado por `makeOutputFile()`.

- **Interfaz de Línea de Comandos (`cli.c/.h`):** En `returnTargetStage()` se estableció `codinter` como la etapa objetivo por defecto cuando no se especifica la bandera `-target`, además de reconocer explícitamente la opción `-target codinter`.

5. Instrucciones de Compilación y Ejecución de Pruebas

El proyecto mantiene la integración con **Ceedling** (Unity + CMock) para la ejecución de pruebas unitarias y la construcción del ejecutable `c-tds`.

5.1. Ejecución de Tests Unitarios con Ceedling

En esta etapa se implementó el archivo de pruebas `test/test_codinter.c`, el cual integra el flujo completo desde el escaneo en memoria (`yy_scan_string`), el parseo (`yyparse`) y la validación semántica (`analyzeAst`), hasta la generación de código intermedio (`generateCodinterFromAST`), verificando la estructura de la lista devuelta por `getCodinterListHead()` e imprimiendo las instrucciones con `printCodinter(stdout)`:

- **Correr tests del Generador de Código Intermedio:**

  ```
  ./ceedling test:codinter
  ```

- **Correr la batería completa de tests del proyecto:**

  ```
  ./ceedling test:all
  ```

5.2. Construcción y Uso del Compilador (Generación de archivos `.ci`)

Para compilar la versión de lanzamiento (*Release*) y ejecutar el compilador hasta la etapa de código intermedio sobre un archivo fuente `.ctds`:

```
./compile.sh
./build/artifacts/release/c-tds -target codinter -o salida prueba.ctds
```

Esto generará el archivo `salida.ci` (o `outputFile.ci` por defecto si se omite `-o`) con la secuencia de instrucciones de tres direcciones.

6. Problemas Conocidos y Testing

Durante el desarrollo y testeo de esta cuarta etapa se identificaron y trabajaron los siguientes aspectos técnicos:

- **Propagación de Tipos en Llamadas a Funciones (`functionInvoke`):** En `parser.y`, los nodos de invocación a funciones se crean inicialmente con `type = blank`. Dado que `generateExprCodinter` evalúa `sym->type != tVoid && sym->type != blank` para decidir si emite una asignación a un temporal (`tX = CALL func`) o una llamada simple (`CALL func`), es importante que el analizador semántico actualice el tipo de retorno sobre el nodo de invocación durante `evaluateFunctionInvoke`.